UNIVERSITY OF ZIMBABWE • DEPARTMENT OF COMPUTER SCIENCE

Clinic Appointment System

Mini Project Report — Programming Languages

Course: Programming Languages
Programme: BSc Honours in Computer Science (HCS)
Authors: Super Washington Banga (R2420381) & Anesu Prosper Matanhire (R2420441)
Language: Python 3.12+ (tested on Python 3.14.4)
Repository: public `clinic-assignment`

Pair submission • Department of Computer Science • November 2026

Contents

Executive Summary 3

1. Introduction and Problem Statement 4

2. Language Choice and the P Parameter 5

3. Architecture and Class Model 6

4. Q1 — Domain Hierarchy 8

5. Q2 — Encapsulation, State Machine and Invariants 9

6. Q3 — Pure Polymorphism for Fees 11

7. Q4 — Deterministic Transaction Logging 12

8. Q5 — Multiple Inheritance and Java Comparison 13

9. Q6 — Inter-Object Rules and All Four Twists 15

10. Q7 — Design Decision Log (D1–D6) 17

11. Testing and Verification 19

12. Q8 — Demo and Live-Change Readiness 20

13. Limitations, Reflection and Conclusion 21

Appendix A — P Arithmetic 22

Appendix B — Demo Output Excerpt 23

Executive Summary

This report documents a deterministic clinic appointment system built for the Programming Languages mini project at the University of Zimbabwe. The system models people, patients, clinical staff, appointments and a clinic aggregate root with a small but complete object-oriented design. It answers all eight assignment questions and provides executable evidence for every behaviour: an 80-test suite, a five-minute demonstration script, and four independently selectable P mod 4 twists.

The implementation deliberately uses Python's context-manager protocol for transactional logging and Python's C3 linearisation with cooperative super() for the MedicalLecturer diamond. It never reads the wall clock: all time is supplied by an injected Clock. Staff fees, staff buffer hours, patient billing multipliers and appointment state transitions are all polymorphic; the production code and tests contain no isinstance/type/hasattr dispatch, no role-string branching and no enum type tags for staff or patient behaviour.

> **Verified arithmetic correction**
>
> The pair's registration numbers are congruent modulo 3, 4 and 5 because their difference is 60 = LCM(3,4,5). The verified shared residue triple is (mod 5, mod 3, mod 4) = (1, 2, 1), not (1, 1, 1). Therefore both partner IDs give Doctor 45 and Nurse 12. The combined sum 4,840,822 has residues (2, 1, 2), giving Doctor 50 and Nurse 11. All tables, tests, slides and scripts use these values.

The report is organized to mirror the brief: Q1–Q8 are answered in order, followed by the D1–D6 design decision log, test evidence, limitations and appendices. The repository is intentionally free of generated documents; this report and the presentation/video guides live in the parent directory.

Evidence at a glance

Check	Result
Test suite	80 unittest tests; also 80 passed under pytest
Coverage	95% of 632 source statements (29 missed)
Twist policies	All four exercised: waiting list, emergency, buffer, discounts
Log frames in demo	49 [log] opened and 49 [log] closed
Forbidden type dispatch in src/main/tests	None
System clock reads in src/main	None
Deliverables	Full report, 15-slide deck, three video guides (parent directory)

1. Introduction and Problem Statement

A clinic needs to register patients and staff, book one-hour appointments in a single week (hours 0 to 168), confirm or cancel bookings, enforce scheduling invariants, and write every action to an audit log. The academic exercise uses this domain to test abstraction, encapsulation, state machines, polymorphism, multiple inheritance, exception safety, and the ability to evolve a design under a live change request.

The brief permits Java, Python or C++. Python was selected because the language provides two especially clean mechanisms for the hardest requirements: the with statement (context managers) for Q4 and multiple class inheritance with C3 linearisation for Q5. The rest of the design is deliberately small and explicit.

1.1 Requirements traceability

Question	Requirement	Primary implementation	Evidence
Q1	Person, Patient, Staff, Doctor, Nurse, Pharmacist, Appointment, Clinic + class diagram	src/models/*, src/clinic.py, README diagrams	Class tests; figures 1-2
Q2	Encapsulation, validation, status transitions, invariants	Person.__init__, Appointment, Clinic	State/validation/defensive-copy tests
Q3	Polymorphic fees and one Staff loop, no type checks	Staff.fee(), Doctor/Nurse/Pharmacist	Fee tests, custom Radiologist test
Q4	Exactly-once log around every action and failure	AppointmentLogger, Clinic sessions	80 tests incl. broken sink/file
Q5	Lecturer + MedicalLecturer, one Person, counter proof	src/models/lecturer.py	MRO/counter tests; main.py
Q6	Unique IDs, double-booking rules, P mod 4 twist	Clinic + src/policies.py	All four twist test classes
Q7	Design decision log D1–D6	README and section 9 of this report	Written analysis
Q8	5-minute demo + 10-minute live change	main.py + video guides	Deterministic console output

The project also ships a parent-directory presentation deck and three video guides. The repository itself contains only source, tests, README, pyproject and gitignore.

2. Language Choice and the P Parameter

2.1 Why Python

- **Q4 context management:** __enter__/__exit__ guarantee that [log] opened and [log] closed frame every operation even when a domain exception is raised.
- **Q5 diamond:** Python permits class Lecturer(Person) and Doctor(Staff) to be combined in MedicalLecturer(Doctor, Lecturer); C3 plus cooperative super() gives one Person.__init__ call.
- **Pure polymorphism:** virtual methods on Staff and Patient are simpler and more open than a chain of type checks.
- **Deterministic time:** an injected Clock object removes the wall clock completely.

2.2 P is not defined by the brief

The assignment refers to a parameter P but does not state how it is formed. This project makes the assumption explicit: P defaults to Student 1's registration number, and the Clinic also accepts any non-negative integer p_value. Both partner IDs produce Twist 1; the combined sum produces Twist 2; synthetic values exercise Twists 0 and 3. Staff freeze their P at construction so a fee never changes mid-life; the Clinic's p_value selects the twist policy once.

2.3 Verified modulo derivation

P source	P	P mod 5	Doctor fee	P mod 3	Nurse fee	P mod 4	Twist
Super	2,420,381	1	45	2	12	1	Emergency
Anesu	2,420,441	1	45	2	12	1	Emergency
Combined	4,840,822	2	50	1	11	2	Buffer
Synthetic 0	2,420,380	0	40	1	11	0	Waiting list
Synthetic 3	2,420,383	3	55	1	11	3	Discounts

Worked checks: 3 × 806,793 = 2,420,379, so 2,420,381 mod 3 = 2. 3 × 806,813 = 2,420,439, so 2,420,441 mod 3 = 2. 3 × 1,613,607 = 4,840,821, so the sum mod 3 = 1. The shared-residue property is real; the specific residue values in the initial draft analysis were not. The implementation and all tests use the verified values.

> **Why both partner IDs agree**
>
> 2420441 - 2420381 = 60, and 60 is divisible by 3, 4 and 5. Congruence is therefore preserved for every fee and twist modulus. The agreement is in the residues, not in the claim that each residue equals 1.

3. Architecture and Class Model

The design separates the domain model from orchestration and cross-cutting concerns. Person, Patient, Staff, Doctor, Nurse, Pharmacist, Lecturer and MedicalLecturer carry domain state; Appointment owns its state machine; Clinic owns registration, scheduling rules, the waiting-list policy and the logging boundary. Clock and AppointmentLogger are injected, and TwistPolicy is a small strategy hierarchy.

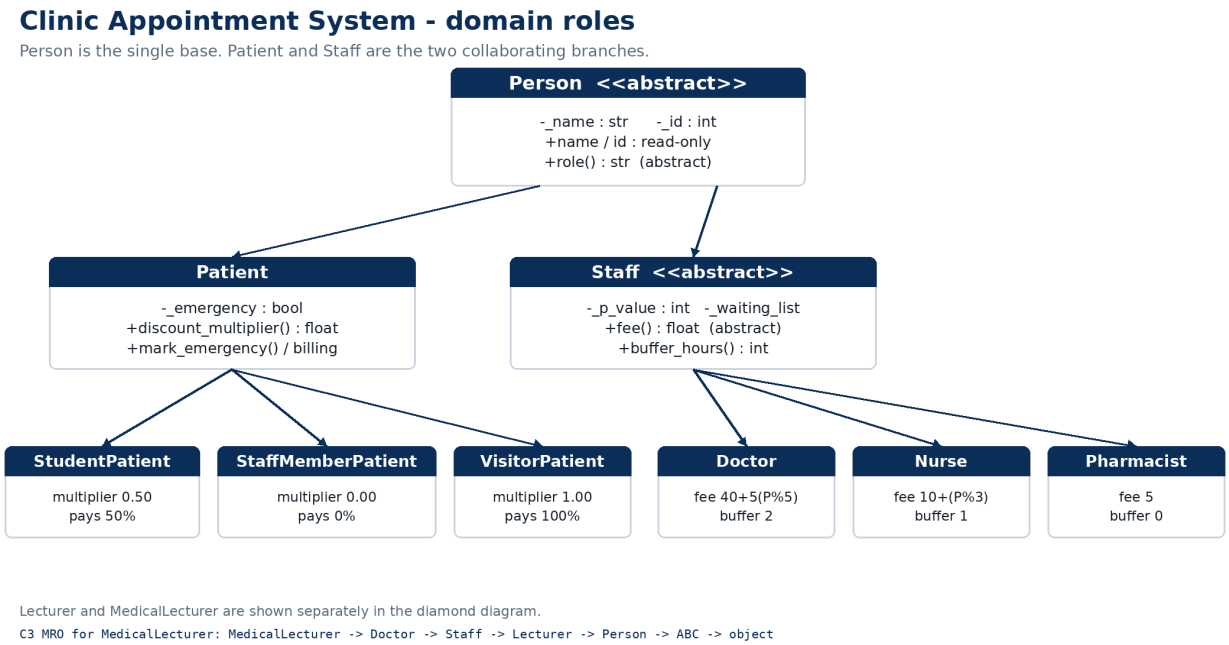


Figure 1 — Person, Patient and Staff role hierarchy.

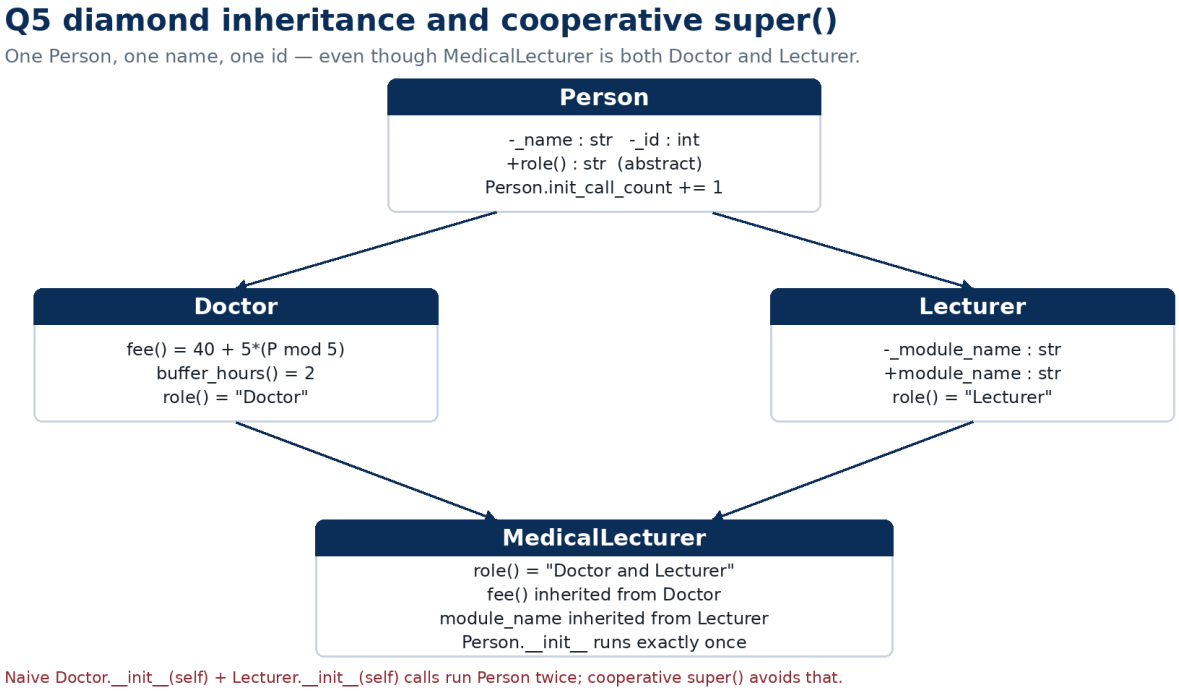


Figure 2 — The Q5 diamond and its C3 MRO.

Clinic collaboration and transaction boundary

Clinic owns the transaction; Clock and Logger are injected; policies delegate to domain objects.

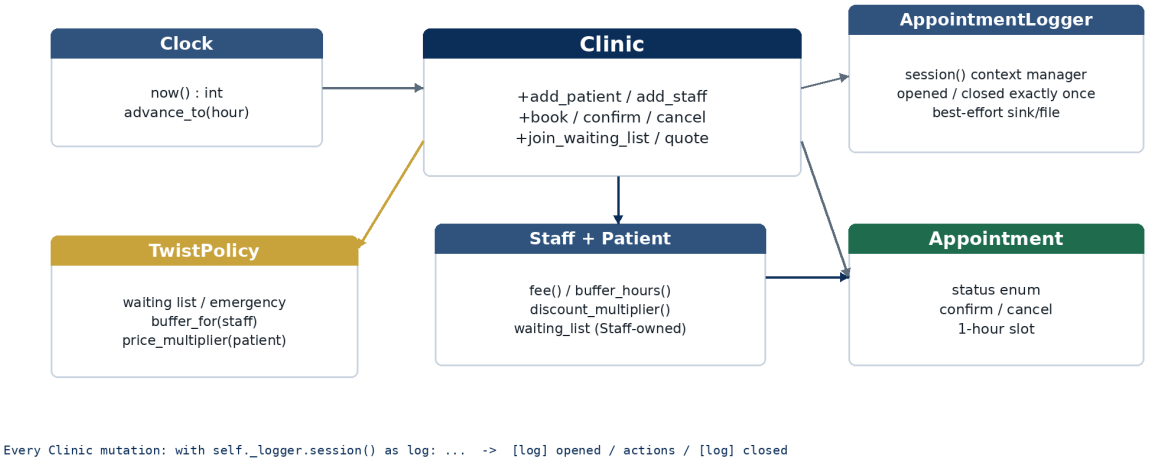


Figure 3 — Clinic collaboration and transaction boundary.

3.1 Package layout

```
src/
  config.py          pair metadata, P helpers, bounds and validation
  clock.py           injected deterministic clock (0..168)
  exceptions.py      ClinicError hierarchy; ValidationError is also ValueError
  logger.py          AppointmentLogger + exactly-once session context
  policies.py        TwistPolicy + four P mod 4 strategies
  clinic.py          aggregate root: IDs, booking, cancellation, offers
  models/
    person.py        abstract Person, name/id validation, init counters
    patient.py       Patient + Student/StaffMember/Visitor billing types
    staff.py         abstract Staff, Doctor, Nurse, Pharmacist, waiting list
    lecturer.py      Lecturer and MedicalLecturer(Doctor, Lecturer)
    appointment.py   AppointmentStatus and the Appointment state machine
main.py              five-minute deterministic demonstration
tests/               80 unittest/pytest-compatible tests
```

4. Q1 — Domain Hierarchy

Person is an abstract base with a private name and id, read-only properties and an abstract role() method. Patient adds an emergency flag and polymorphic billing. Staff is abstract because fee() is role-specific; it adds P, buffer_hours() and a per-staff waiting list. Doctor, Nurse and Pharmacist are concrete staff roles. Lecturer is a Person with a module name. MedicalLecturer inherits from both Doctor and Lecturer. Appointment links one patient to one staff member at one start hour. Clinic is the aggregate root.

The Mermaid class diagram in README.md renders natively on GitHub. The report contains the same model as a rendered figure and as the ASCII UML fallback.

4.1 Person and constructor validation

```
class Person(ABC):
    """A named, uniquely identified participant in the clinic.

    Class invariants
    ----------------
    P1. ``_id`` is a positive integer.
    P2. ``_name`` is a non-empty, whitespace-trimmed string.
    P3. ``role()`` is implemented by every concrete subclass.
    P4. ``Person.init_call_count`` increments once for every completed
        ``Person.__init__`` call; ``person_init_calls`` records the number of
        times this particular instance's Person initialiser ran.
    """

    # Class-level counter used by the Q5 / D4 proof.
    init_call_count = 0

    # Class default; each instance shadows it with its own value on first init.
    person_init_calls = 0

    def __init__(self, name: object, id: object, **kwargs: object) -> None:
        self._name = validate_name(name, "name")
        identifier = require_int(id, "id")
        if identifier <= 0:
            raise ValidationError("id must be a positive integer")
        self._id = identifier
        self.person_init_calls = self.person_init_calls + 1
        Person.init_call_count += 1
        # Continue the cooperative chain (normally to ABC/object).
        super().__init__(**kwargs)
```

Validation uses operator.index() rather than isinstance/type/hasattr. ValidationError inherits from both ClinicError and ValueError, so the brief's requirement to raise ValueError is satisfied while the Clinic can catch one domain root.

5. Q2 — Encapsulation, State Machine and Invariants

5.1 Encapsulation

Every instance field is underscore-prefixed. Collections are returned as fresh tuples from Clinic.appointments, Clinic.staff, Clinic.patients, Staff.waiting_list, Clinic.waiting_list_for and AppointmentLogger.entries. A caller can append to the returned tuple only by causing an AttributeError; the internal list is untouched. The test suite exercises this for every exposed collection.

5.2 Legal transitions

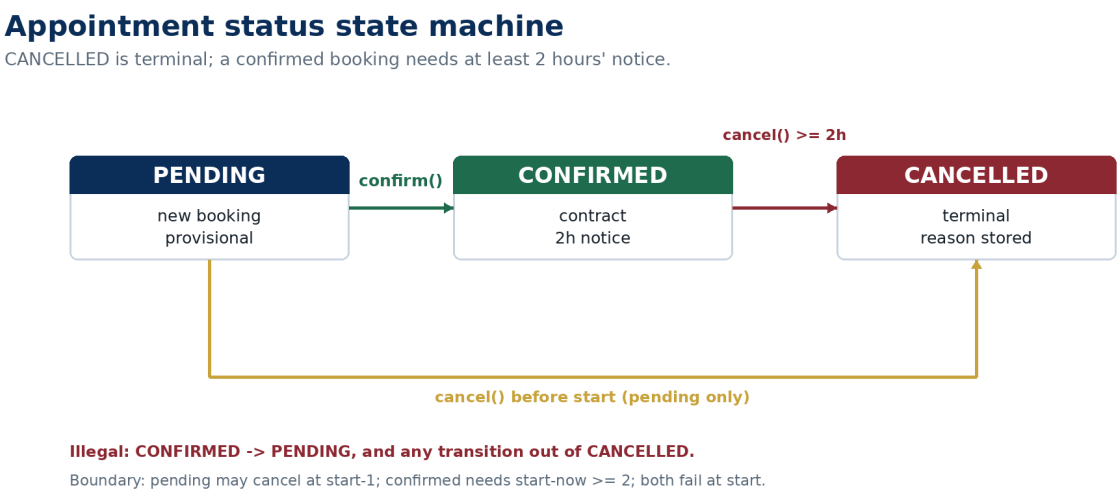


Figure 4 — Appointment status machine.

From	To	Condition	Result
PENDING	CONFIRMED	confirm()	Allowed
PENDING	CANCELLED	now < start	Allowed; no 2-hour notice requirement
CONFIRMED	CANCELLED	start - now >= 2	Allowed
CONFIRMED	CANCELLED	start - now < 2	CancellationError('too late')
PENDING/CONFIRMED	CANCELLED	now >= start	CancellationError('too late')
CANCELLED	any	always	InvalidTransitionError (terminal)

5.3 The cancellation rule

```
    def cancel(self, current_hour: object, reason: object = "patient request") -> None:
        """Move PENDING/CONFIRMED -> CANCELLED, enforcing all timing rules."""

        now = validate_hour(current_hour, "current_hour")
        cleaned_reason = validate_name(reason, "cancellation reason")

        if self._status is AppointmentStatus.CANCELLED:
            raise InvalidTransitionError("already cancelled")
        if now >= self._start_hour:
            raise CancellationError("too late")
        if (
            self._status is AppointmentStatus.CONFIRMED
            and self._start_hour - now < CANCELLATION_NOTICE_HOURS
        ):
            raise CancellationError("too late")

        self._status = AppointmentStatus.CANCELLED
        self._cancellation_reason = cleaned_reason
```

The 2-hour rule lives in Appointment.cancel, not Clinic. The appointment owns its start hour and status, so the rule cannot be bypassed by calling a different public method. Clinic passes clock.now(); the policy is therefore testable at exact boundaries without sleeping. Section 9 (D1) discusses the alternatives.

5.4 Invariants

Formal invariant blocks appear directly above class Appointment and class Clinic in the source. Appointment invariants cover id positivity, start-hour bounds, valid status, terminal CANCELLED, legal transitions, the two-hour notice rule, the started rule, cancellation-reason consistency and stable participant references. Clinic invariants cover a single ID namespace, exact registered participants, no staff/patient double-booking, no confirmed bump, buffer enforcement under Twist 2, waiting-list uniqueness, one log session per mutation and an immutable twist policy.

6. Q3 — Pure Polymorphism for Fees

The three fee formulas are the only place the P-dependent arithmetic is written: Doctor 40 + 5*(P mod 5), Nurse 10 + (P mod 3), Pharmacist 5. The demonstration and tests iterate over Clinic.staff, which is a tuple of Staff, and call virtual role() and fee() methods in one loop.

```
# src/config.py
def doctor_fee_for(p_value):
    return 40.0 + 5.0 * p_mod(p_value, 5)

def nurse_fee_for(p_value):
    return 10.0 + float(p_mod(p_value, 3))

# the one heterogeneous loop (main.py / README)
for member in clinic.staff:
    print(f"{member.role():<12} fee = {member.fee():.2f}")
```

With P = 2,420,381 the output is Doctor 45.00, Nurse 12.00, Pharmacist 5.00. With P = 4,840,822 it is Doctor 50.00, Nurse 11.00, Pharmacist 5.00. There is no isinstance, type(), hasattr, role-string comparison or enum type tag in the loop or in Clinic. The test suite adds a Radiologist subclass with fee 70 and buffer 3; it is accepted by the existing Clinic and loop without any production edit. This is the Open/Closed evidence for D3.

P	Doctor	Nurse	Pharmacist	Active twist
2420381	45.00	12.00	5.00	Emergency (mod 4 = 1)
2420441	45.00	12.00	5.00	Emergency (mod 4 = 1)
4840822	50.00	11.00	5.00	Buffer (mod 4 = 2)

7. Q4 — Deterministic Transaction Logging

AppointmentLogger implements __enter__ and __exit__. Clinic wraps every public mutation in exactly one with self._logger.session() as log: block. The session emits [log] opened on entry and [log] closed on exit, even when a domain exception is active. A nested session is rejected, which prevents accidental duplicate framing. Failures are recorded before the exception is re-raised. The exact failure line has no [log] prefix, matching the brief:

```
[log] opened
FAILED cancel #3: too late
[log] closed
```

The logger records each entry in memory first and returns a defensive tuple from entries. It then attempts the injected sink and optional file. A broken sink or file is reported to stderr but never replaces the primary domain exception and never leaves the session open. The test suite injects an OSError sink and an invalid file path to verify both guarantees.

```
class _LogSession:
    """One context-managed logging transaction."""

    def __init__(self, logger: "AppointmentLogger") -> None:
        self._logger = logger
        self._closed = False

    def __enter__(self) -> "_LogSession":
        if self._logger._active:
            raise RuntimeError("nested appointment-log sessions are not allowed")
        self._logger._active = True
        try:
            self._logger._emit("[log] opened")
        except Exception:
            self._logger._active = False
            raise
        return self

    def write(self, message: object) -> None:
        """Record an action or failure line."""

        self._logger._emit(str(message))

    def __exit__(self, exc_type, exc, traceback) -> bool:
        if self._closed:
            return False
        self._closed = True
        try:
            self._logger._emit("[log] closed")
        finally:
            self._logger._active = False
        # Never suppress the domain exception that caused the exit.
        return False
```

7.1 Logged actions

Successful registrations, bookings, confirmations, cancellations, waiting-list joins, offers and bumps are logged. Failed registration, booking, confirmation, cancellation and waiting-list actions are logged with the exact FAILED form before re-raising. The cancel operation may also create a waiting-list offer inside the same session, so the operation still has one opened/closed pair even when it performs two actions.

8. Q5 — Multiple Inheritance and Java Comparison

MedicalLecturer(Doctor, Lecturer) has the MRO MedicalLecturer -> Doctor -> Staff -> Lecturer -> Person -> ABC -> object. MedicalLecturer.__init__ forwards name, id, module_name and p_value into cooperative super(). Doctor forwards module_name in **kwargs, Staff consumes p_value, Lecturer consumes module_name, and Person runs once. The class counter Person.init_call_count and the instance counter person_init_calls both show 1.

```
class MedicalLecturer(Doctor, Lecturer):
    """A clinician who also lectures.

    MRO: MedicalLecturer -> Doctor -> Staff -> Lecturer -> Person -> ABC.
    The cooperative ``super()`` chain consumes each class's arguments and calls
    Person.__init__ exactly once.
    """

    def __init__(
        self,
        name: object,
        id: object,
        module_name: object,
        p_value: object = None,
        **kwargs: object,
    ) -> None:
        super().__init__(
            name=name,
            id=id,
            module_name=module_name,
            p_value=p_value,
            **kwargs,
        )

    def role(self) -> str:
        return "Doctor and Lecturer"
```

8.1 What does MedicalLecturer.fee() return?

It inherits Doctor.fee(), which is 45.00 with the default P. A clinical appointment is charged at the medical consultation rate; the teaching role adds a module responsibility but does not replace the clinical fee. C3 resolves Doctor.fee before any Lecturer member because Doctor precedes Lecturer in the MRO. Lecturer does not define fee().

8.2 The naive double-initialisation bug

If MedicalLecturer called Doctor.__init__(self, ...) and Lecturer.__init__(self, ...) explicitly, the cooperative Doctor->Staff->Lecturer->Person chain would already have initialised Person, and the second explicit call would initialise it again. The test suite contains a NaiveMedicalLecturer that demonstrates Person.init_call_count delta 2 and person_init_calls 2. In a richer hierarchy this would duplicate side effects and overwrite shared state.

8.3 Java restructure

Java forbids multiple class inheritance. Lecturer must become an interface with getModuleName(); MedicalLecturer extends Doctor implements Lecturer and must duplicate the moduleName field and accessor because interfaces cannot store per-instance state. The role() override remains. Python's inheritance of the single Lecturer field and one Person initialisation is the structural advantage.

```
interface Lecturer {
    String getModuleName();
}

class MedicalLecturer extends Doctor implements Lecturer {
    private final String moduleName;          // storage duplicated
    MedicalLecturer(String name, int id, String moduleName) {
        super(name, id);
        this.moduleName = moduleName;
    }
    public String getModuleName() { return moduleName; }
    public String role() { return "Doctor and Lecturer"; }
}
```

9. Q6 — Inter-Object Rules and All Four Twists

9.1 Core rules

- One ID namespace: Clinic._used_ids covers registered persons and appointments. Auto-generated appointment IDs skip any person ID already in use.
- No staff member has two non-cancelled appointments at one hour.
- No patient has two non-cancelled appointments at one hour, even with different staff.
- A cancelled appointment remains in the audit history but no longer conflicts.
- Every public mutation produces exactly one log session; failures are logged before re-raising.

9.2 Twist 0 — waiting list

Staff owns its waiting list because the queue is per-staff; Clinic coordinates the offer because only Clinic sees all appointments and conflicts. When a cancellation occurs under WaitingListPolicy, Clinic scans the queue in arrival order and offers the slot to the first registered patient with no patient conflict, no staff conflict, no buffer conflict, and who is not the patient who just cancelled. The offer is a new PENDING appointment with a new ID; the patient is removed only after creation; the offer is logged. If no one is eligible, the queue is preserved.

9.3 Twist 1 — emergency bump

A patient can be flagged with mark_emergency(). Under EmergencyBumpPolicy, an emergency booking that meets a PENDING appointment at the same staff/hour cancels that appointment with reason 'bumped' and creates the new booking. A CONFIRMED appointment is never bumped; the booking fails and is logged. Patient conflicts are checked before bumping so an emergency cannot destroy an unrelated PENDING slot and then fail. A started PENDING appointment cannot be bumped because cancellation would violate the started rule.

```
    @staticmethod
    def _free_gap_hours(existing_hour: int, new_hour: int) -> int:
        """Free hours between two one-hour appointments.

        If the earlier appointment starts at ``h`` it occupies ``[h, h+1)``.
        The gap to the later start is therefore ``later - (earlier + 1)``.
        Equality with the staff buffer is allowed.
        """

        if existing_hour < new_hour:
            return new_hour - (existing_hour + APPOINTMENT_DURATION_HOURS)
        return existing_hour - (new_hour + APPOINTMENT_DURATION_HOURS)

    def _find_buffer_conflict(self, staff: Staff, hour: int) -> Appointment | None:
        required_gap = self._policy.buffer_for(staff)
        if required_gap <= 0:
            return None
        for appointment in self._appointments.values():
            if not appointment.is_active or appointment.staff.id != staff.id:
                continue
            if self._free_gap_hours(appointment.start_hour, hour) < required_gap:
                return appointment
        return None
```

9.4 Twist 2 — buffer

Clinic._free_gap_hours is the single buffer helper. A one-hour appointment at h occupies [h, h+1). A staff member with buffer b requires the later start to be at least earlier_start + 1 + b. A doctor at 10 may therefore next start at 13, not 12. Clinic asks staff.buffer_hours(); Doctor returns 2, Nurse 1, Pharmacist 0, and a custom subclass can return its own value. The policy only calls this method under

Twist 2, so buffer behaviour is dormant in the other twists.

9.5 Twist 3 — patient discounts

Patient category	discount_multiplier()	Meaning
StudentPatient	0.50	pays 50% of staff fee
StaffMemberPatient	0.00	pays 0%
VisitorPatient / Patient	1.00	pays 100%

Clinic.quote composes staff.fee() * policy.price_multiplier(patient). Under DiscountPolicy the multiplier is patient.discount_multiplier(); otherwise it is 1.0. The discount lives on Patient because it is a patient-category property, not a staff property. A senior category needs only a new Patient subclass.

9.6 Switching the active twist

```
Clinic(clock)                           # DEFAULT_P 2420381 -> Twist 1
Clinic(clock, p_value=STUDENT_2_ID)      # 2420441          -> Twist 1
Clinic(clock, p_value=COMBINED_PAIR_P)   # 4840822          -> Twist 2
Clinic(clock, p_value=DEMO_P_TWIST_0)    # synthetic        -> Twist 0
Clinic(clock, p_value=DEMO_P_TWIST_3)    # synthetic        -> Twist 3
```

10. Q7 — Design Decision Log (D1–D6)

D1. Where does the 2-hour rule live?

Decision: Appointment.cancel(current_hour). The appointment owns its start hour and status; Clinic supplies clock.now(). This is Information Expert (GRASP) and gives high cohesion. The main trade-off is that a per-role notice window needs a parameter or policy; the live-change guide demonstrates that extension.

- **Clinic:** central orchestration, easy global change, but the rule leaks away from the state it protects and is bypassable by direct Appointment.cancel().
- **Staff:** natural for per-role windows, but staff do not own appointment status or start hours; a back-reference and blurred responsibility result.
- **Separate policy object:** most configurable and independently testable, but adds indirection and another source of truth for one global rule.

D2. Why is the clock design testable?

Clock is injected; the domain never imports time or datetime. Tests jump instantly to start-3, start-2, start-1, start and 168. If the code called the system clock directly, tests would need to sleep, would be flaky around real-time changes, and could not reproduce the exact two-hour/started boundaries. The demo is deterministic for the same reason.

D3. When Pharmacist was added, what leaked?

Nothing in production code. Pharmacist implements role() and fee(); it inherits buffer_hours() 0. Clinic.add_staff already accepts any Staff, and the Q3 loop calls virtual methods. The only edits were registration in demo/test code and documentation. The test suite proves the stronger case by adding a Radiologist with a custom fee and buffer entirely inside the tests.

D4. MedicalLecturer.fee() and double-initialisation prevention

It returns Doctor.fee() (45.00 default) because the clinical consultation rate applies. Python's C3 linearisation plus cooperative super() ensures Person.__init__ runs once. Without cooperation, a naive Doctor.__init__ + Lecturer.__init__ would run Person twice (delta 2 in the test), duplicating constructor side effects. Java would make Lecturer an interface and force MedicalLecturer to duplicate moduleName storage.

D5. What if writing to or closing the log fails?

Logging is best-effort and must never corrupt or mask the domain operation. The logger records in memory first, then tries the sink and optional file. OSError and display failures go to stderr. __exit__ always attempts [log] closed once and returns False, so the primary exception propagates. A log-file failure does not roll back a successful booking or cancellation. In an audit-critical deployment one could choose fail-closed, but even then the logger must not replace the domain exception. The suite injects a failing sink and a directory as the log path to prove both guarantees.

D6. Twist-specific answers

(0) Waiting-list ownership. Staff owns the list because the queue is per-staff and staff is the Information Expert for demand on its own time. Clinic coordinates joining, conflicts and offers because only Clinic sees all appointments and owns the transaction log. Clinic ownership would bloat the aggregate; Patient ownership would fragment per-staff order.

(1) Why confirmed cannot be bumped. Confirmation is a domain contract: patient and staff have committed. Allowing displacement would violate trust and make confirmation meaningless. Emergency

privilege is deliberately limited to PENDING provisional bookings; the failed attempt is logged.

(2) Why buffer_hours() and not a type check. The staff member knows its own clinical scheduling needs. A virtual method keeps that knowledge local, satisfies Open/Closed, and lets a new Radiologist/Physiotherapist declare a different buffer without editing Clinic. A type check would force Clinic to know every staff class and leak each new role into central scheduling code.

(3) Where the discount lives. On Patient, as discount_multiplier(). It is a patient-category property, not a staff fee. Clinic composes the independent pieces: price = staff.fee() * patient.discount_multiplier(). Staff does not know patient billing categories; Patient does not know staff fees. A senior category is a new Patient subclass with no Clinic edit.

11. Testing and Verification

The repository contains 80 unittest tests, also collected by pytest. The suite covers clock bounds and forward-only movement; constructor validation; defensive copies; every legal and illegal status transition; the exact start-3/start-2/start-1/start boundaries; the exact FAILED cancel #3: too late format; logging success/failure, nested-session rejection and broken sink/file handling; unique IDs, staff/patient double-booking, all four twists, waiting-list conflicts, buffer boundaries, custom staff/patient subclasses, C3 MRO and the naive double-init counter.

```
$ python3 -m unittest discover -s tests -v
...
Ran 80 tests in 0.006s

OK

$ pytest -q
80 passed, 23 subtests passed in 0.07s
```

Coverage measured with coverage.py over src/ is 95% (632 statements, 29 missed). The remaining misses are mostly defensive branches and display fallbacks, not untested public behaviour. Both unittest and pytest exit 0.

Module	Coverage	Notes
src/clinic.py	95%	defensive branches / fallback paths
src/logger.py	90%	sink/file fallback paths
src/clock.py	95%	one boundary guard
src/models/appointment.py	98%	repr path
src/models/person.py	96%	str path
src/policies.py	97%	repr path
TOTAL	95%	632 source statements, 29 missed

11.1 Demo and static checks

python3 main.py exits 0 and prints both partners' names and registration numbers, Q1–Q6, all four twists, the MRO counter proof and deliberate failures. The output contains one [log] opened and one [log] closed per action, 49 opened and 49 closed in the full demo, and the exact lines FAILED cancel #3: too late and FAILED book: staff conflict at hour 14. Static checks confirm no isinstance/type/hasattr or __class__ dispatch in src, main.py or tests, and no time/datetime imports in src or main.py.

12. Q8 — Demo and Live-Change Readiness

The required five-minute demo is main.py. The companion video guide gives the minute-by-minute split for Super Washington Banga and Anesu Prosper Matanhire. The demo sections map directly to the brief:

Section	What is shown	Deliberate evidence
Q1/Q2	Hierarchy, roles, booking, confirmation, defensive copy, validation	Tuple append fails; empty name raises
Q3	One staff loop	Doctor 45.00, Nurse 12.00, Pharmacist 5.00
Q4	Transaction log and failures	FAILED cancel #3: too late; staff conflict
Q5	Diamond and counter	Person.__init__ delta 1; MRO
Q6	All four twists	Bump, buffer, waiting list, discounts
Close	Test commands and pair metadata	80 tests passing

The live-change guide rehearses six likely requests: add a Radiologist, add NO_SHOW, make cancellation notice per staff role, add a daily cap, add a recurring series, and add a senior-patient category. Each has an exact diff, test snippet, driver/navigator split and a short explanation. The diffs are applied and tested in a throwaway copy before the guide is finalized.

The 10-minute protocol is: 0:00–1:00 read the request and restate acceptance; 1:00–2:00 locate the extension point and write the test; 2:00–6:00 implement; 6:00–8:00 run the full suite and the demo; 8:00–10:00 explain the design and the test evidence. The navigator watches for type checks, missing logging, and untested boundaries.

13. Limitations, Reflection and Conclusion

13.1 Limitations

- P is not defined by the brief; the pair-ID assumption is documented and configurable, and all four policies are implemented.
- Buffer is interpreted as free time after the one-hour appointment. The single _free_gap_hours helper is the only place to change if start-to-start separation is required.
- start_hour = 168 is accepted literally for rubric compatibility; a strict week would use 167 as the final start. end_hour is a derived label.
- There is no persistence, authentication, UI, concurrency or real calendar integration; this is a deterministic domain-model project.
- Booking in the past/current hour is not prohibited because the brief restricts only cancellation. A production system would add a booking-time policy.
- Waiting lists exist in every Clinic but automatic offers only occur under Twist 0; this is deliberate policy gating.

13.2 Reflection

The project demonstrates that the hard parts of the brief are design problems, not syntax problems. Keeping the two-hour rule inside Appointment, keeping fee/buffer/discount knowledge inside the object that owns it, and keeping logging as a surrounding transaction rather than a scattered set of print statements produced a system that is easier to test and easier to change. The P ambiguity was handled by making the assumption explicit and by implementing all four policies instead of guessing one.

The pair also learned to separate submission artefacts from the code repository: the report, slides and video guides are generated outside the repository, while the repository remains a clean, dependency-free Python project with an atomic commit history.

13.3 Conclusion

All eight assignment questions are answered in code and in the documentation. The system enforces encapsulation, validation, the appointment state machine, unique IDs, staff/patient double-booking rules, exactly-once context-managed logging and all four P mod 4 twists. It proves the diamond MRO counter, demonstrates pure polymorphism, and passes 80 tests under both unittest and pytest. The repository is ready for submission and the parent-directory materials support the five-minute and ten-minute assessment formats.

Appendix A — P Arithmetic

The following table is the authoritative arithmetic used by the code, tests, slides and video scripts. The residue correction is deliberate and verified.

Expression	Calculation	Result
2420381 mod 5	last digit 1	1
2420381 mod 3	digit sum 20; 20 mod 3	2
2420381 mod 4	last two digits 81; 81 mod 4	1
2420441 mod 5	last digit 1	1
2420441 mod 3	digit sum 17; 17 mod 3	2
2420441 mod 4	last two digits 41; 41 mod 4	1
4840822 mod 5	last digit 2	2
4840822 mod 3	digit sum 28; 28 mod 3	1
4840822 mod 4	last two digits 22; 22 mod 4	2

The pair's IDs agree because 60 is a multiple of 3, 4 and 5. The sum residues are not all 2: mod 3 is 1. This is why the combined-P nurse fee is 11, not 12.

Fee implications

P	Doctor 40+5(P%5)	Nurse 10+(P%3)	Pharmacist	P mod 4
2,420,381	45	12	5	1 Emergency
2,420,441	45	12	5	1 Emergency
4,840,822	50	11	5	2 Buffer
2,420,380	40	11	5	0 Waiting list
2,420,383	55	11	5	3 Discounts

Appendix B — Demo Output Excerpt

The following is a line-numbered selection from the deterministic main.py run. The full run exits 0 and contains the expected closed markers.

```
010: Q1/Q2 - domain hierarchy, encapsulation and the state machine
044: Q3 - one loop over Staff; role() and fee() dispatch polymorphically
057:     Nurse        fee = 12.00
062: Q4 - [log] opened / [log] closed, including a deliberate failure
087: FAILED cancel #3: too late
095: FAILED book: staff conflict at hour 14
101: Q5 - Lecturer + Doctor diamond, cooperative super() and MRO
107:   Person.__init__ delta: 1
112: Q6 Twist 1 (P mod 4 = 1) - emergency bump of a PENDING appointment
151: Q6 Twist 2 (P mod 4 = 2) - polymorphic staff buffer hours
186: Q6 Twist 0 (synthetic P mod 4 = 0) - waiting-list offer
206: [log] waitlist offered #2 to patient=#6002 for staff=#5001 at hour=20 (PENDING)
212: Q6 Twist 3 (synthetic P mod 4 = 3) - polymorphic patient discounts
227:   Student       multiplier=0.50 -> price=27.50
232: Verification
```

The full console output is retained outside the repository as a generation artefact; the committed main.py is the reproducible source of this evidence.